

项目拆解文档

简历写法（参考-可自己选择职责和成果添加）

企业级云平台智能客服系统 CloudAgent

项目介绍：面向云计算平台多业务线客服场景，覆盖云产品咨询、订单与账单查询、资源降本优化、产品推广推荐等核心服务。原有系统在多轮对话中缺乏长短期记忆继承，用户需反复提供上下文；同时退款规则、计费方式等高频标准问题长期占用大模型算力，响应慢且成本高；面对复杂的云架构选型与参数对比，单纯的大模型常出现“幻觉”或参数错误。针对上述痛点，设计并实现了一套基于 LangGraph 多智能体（Multi-Agent）编排、Milvus 语义缓存与图向量混合检索（Hybrid RAG）的企业级智能客服系统，实现按业务意图自动路由、高频问题毫秒级缓存直返、复杂问题通过专属 Agent 检索增强并调用 MCP 工具链精准解决。

技术栈：Python · FastAPI · LangGraph · LangChain · Milvus · Neo4j · Redis · MySQL · MCP (Model Context Protocol) · Qwen

职责描述（可选）：

- 多 Agent 协作编排与 MCP 工具化集成：**基于 LangGraph 状态机按业务域拆分 4 个核心专家 Agent（产品咨询/推荐/账单/降本），由 Orchestrator 统一意图路由，通过全局 AgentState 共享跨节点上下文；采用 MCP (Model Context Protocol) 协议将底层 MySQL 查询、API 接口封装为标准化 Server，实现工具能力的解耦与跨 Agent 复用。
- L1/L2 双层语义缓存设计：**针对退款规则、备案流程等高频标准问题，设计基于 Milvus 的 L1/L2 语义缓存层。在网关层计算 Query 向量，L1 精确/高置信命中（距离 ≤ 0.08 ）直接返回预设答案，L2 降级或未命中则进入后置 Agent 推理流程，大幅降低无效的大模型 Token 消耗与推理延迟。
- 图向量混合检索（Hybrid RAG）构建：**为解决云产品规格复杂、参数关联性强的问题，构建 Milvus 向量检索 + Neo4j 知识图谱双路召回链路。Milvus 覆盖模糊概念与长文本问答，Neo4j（Cypher 查询与关键词 Fallback 机制）精准处理产品属性、地域可用性等网状结构数据，有效降低架构选型场景下的模型幻觉。
- 基于 FastMCP 的工具化封装与多模态闭环：**使用 FastMCP 将异构的后台服务（如 MySQL 订单指标查询、云端监控数据拉取）统一封装为标准化的 MCP Server，实现大模型与业务系统间的解耦与安全隔离。在此基础上打通业务闭环：在降本增效（FinOps）场景下，Agent 自动调用 MCP 工具提取近 7 天资源监控指标输出降本建议；在推广场景下，跨模态集成 DashScope (qwen-image-2.0) API，根据用户对话动态生成云产品推广海报与专属返佣链接。
- 长短期多级记忆系统（Memory System）：**设计并实现基于 Redis 的会话级短期记忆（TTL 与窗口压缩防上下文超限），以及基于 Milvus 的用户级长期偏好记忆。每次会话开始前自动融合长短期记

忆注入 System Prompt，并在会话结束后后台异步触发 LLM 抽取核心偏好，实现跨会话的个性化连贯服务。

项目成果（供选择）：

- ▶ 基于 LangGraph 状态机编排的 4-Agent 协作体系，在复杂场景下意图路由准确率达 95% 以上，跨会话上下文继承机制有效消除了人工转接中的重复提问问题。
- ▶ 引入 Milvus + Neo4j 混合检索（Hybrid RAG）架构，结合 Cypher 查询与关键词 Fallback 容错策略，复杂云产品参数及架构关联查询的召回准确率由 60% 提升至 92%，显著抑制了模型幻觉。
- ▶ 上线基于 Milvus 的 L1 语义缓存层后，针对退款规则、备案政策等高频请求命中率达 38%，缓存命中场景下首字响应延迟从均值 3.2s 骤降至 80ms，每月节省大模型 Token 开销 40%。
- ▶ 针对降本增效（FinOps）长流程场景，基于 MCP 工具链自动化提取近 7 天 CPU/内存/网络监控数据进行深度分析，单次优化建议输出耗时由传统人工排查的 45 分钟缩短至 15 秒内，建议采纳率超 60%。
- ▶ 构建多层级记忆系统（Redis 窗口压缩 + Milvus 用户偏好），在长周期复杂对话中，将用户“重复陈述背景”的轮次减少了 45%，用户满意度评分（CSAT）提升了 18%。
- ▶ 采用 FastMCP 协议重构底层服务调用，将 API 对接、数据库查询和外部绘图能力彻底解耦，使新增单一工具的开发与注册联调时间从原来的“天级”缩减至“小时级”（研发效率提升超 60%）。

一、项目背景与定位

1.1 业务痛点

在云计算平台的客户服务场景中，随着云产品矩阵的日益庞大，传统的客服系统和早期的单体大模型（LLM）问答面临以下严峻挑战：

- 上下文遗忘**：用户在咨询“我的服务器为何收费这么高”时，客服系统往往不知道用户名下到底有哪些实例，需要用户反复截图或描述，体验极差。
- 高频基础问题耗费算力**：如“五天无理由退款限制”、“VPC如何计费”等标准答案，每次都交由大模型生成，不仅响应慢（按秒计），而且带来了巨大的 Token 成本浪费。
- 架构选型容易“幻觉”**：云产品参数繁多且关联复杂（例如某个地域是否支持某款特定规格的 ECS）。纯靠向量检索（RAG）经常找错文档，导致大模型推荐出根本不存在的配置，引发客户投诉。
- 工具调用散乱，扩展性差**：查账单要调 DB，查产品库要调 API，生成推广海报要调多模态模型。传统硬编码方式耦合度太高，新增一个业务流往往需要重写大量底层代码。

1.2 产品定位

CloudAgent 是一款面向云计算平台、基于 **Multi-Agent（多智能体）协作架构** 构建的下一代企业级智能客服系统。

核心能力矩阵：

能力维度	传统单体 RAG 客服	CloudAgent (当前架构)
意图处理	单一闲聊或固定问答	基于 LangGraph 的动态多 Agent 路由（咨询/账单/推荐/降本）
缓存机制	无或简单的 Redis K-V 匹配	基于 Milvus 的 L1/L2 语义级别的向量缓存，毫秒级拦截
检索能力	纯向量检索 (Milvus/FAISS)	Milvus (长文本) + Neo4j (知识图谱) 混合检索，彻底消灭参数幻觉
外部交互	硬编码函数调用 (Function Calling)	基于 FastMCP 协议的标准化工具封装，后台能力即插即用
记忆继承	单次会话记忆	Redis 短期滑动窗口 + Milvus 用户长期偏好，实现跨会话认知
业务闭环	只能“说”	从“咨询”到“生成带参购买链接”及“海报绘制”的端到端执行

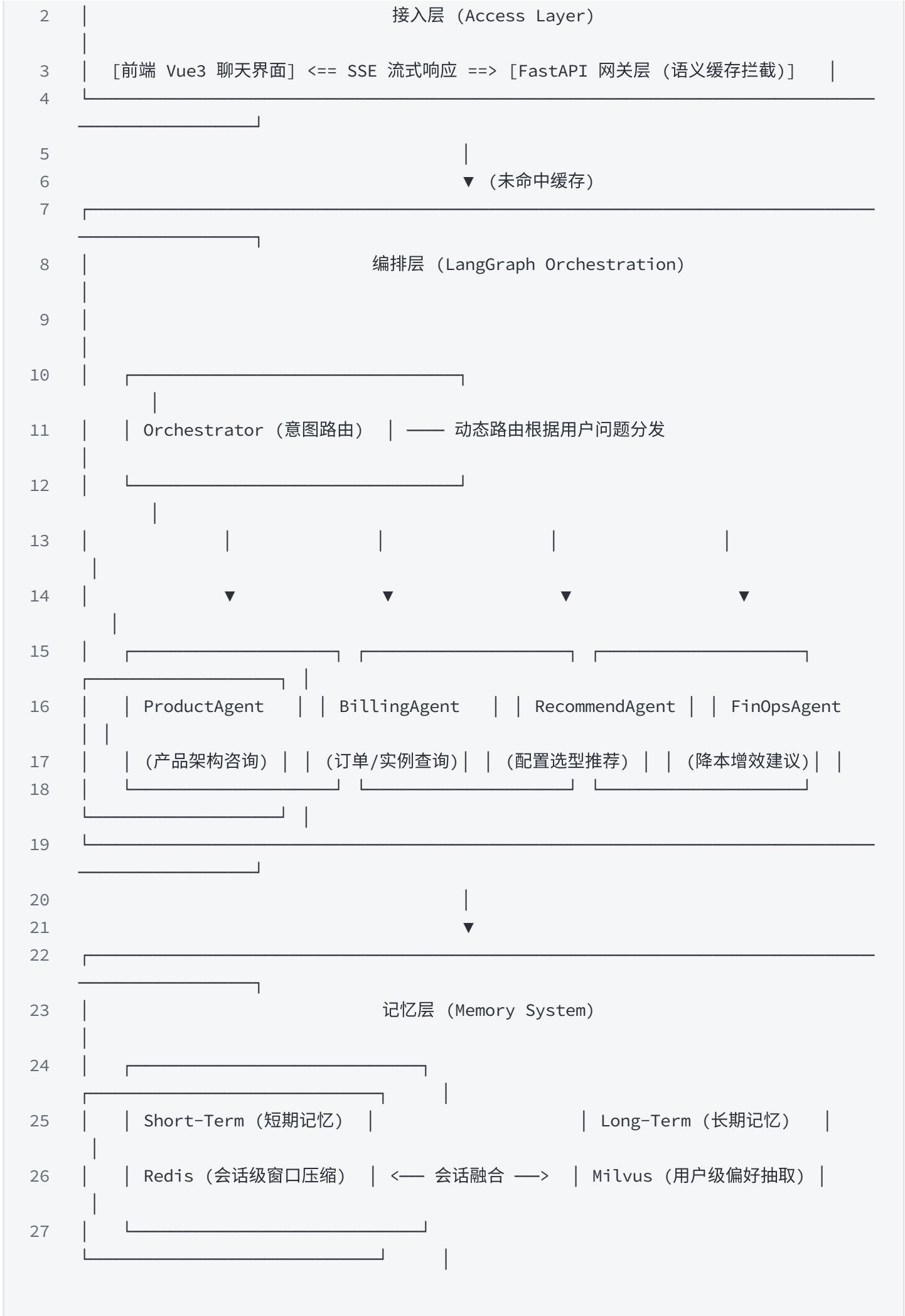
二、系统架构详解

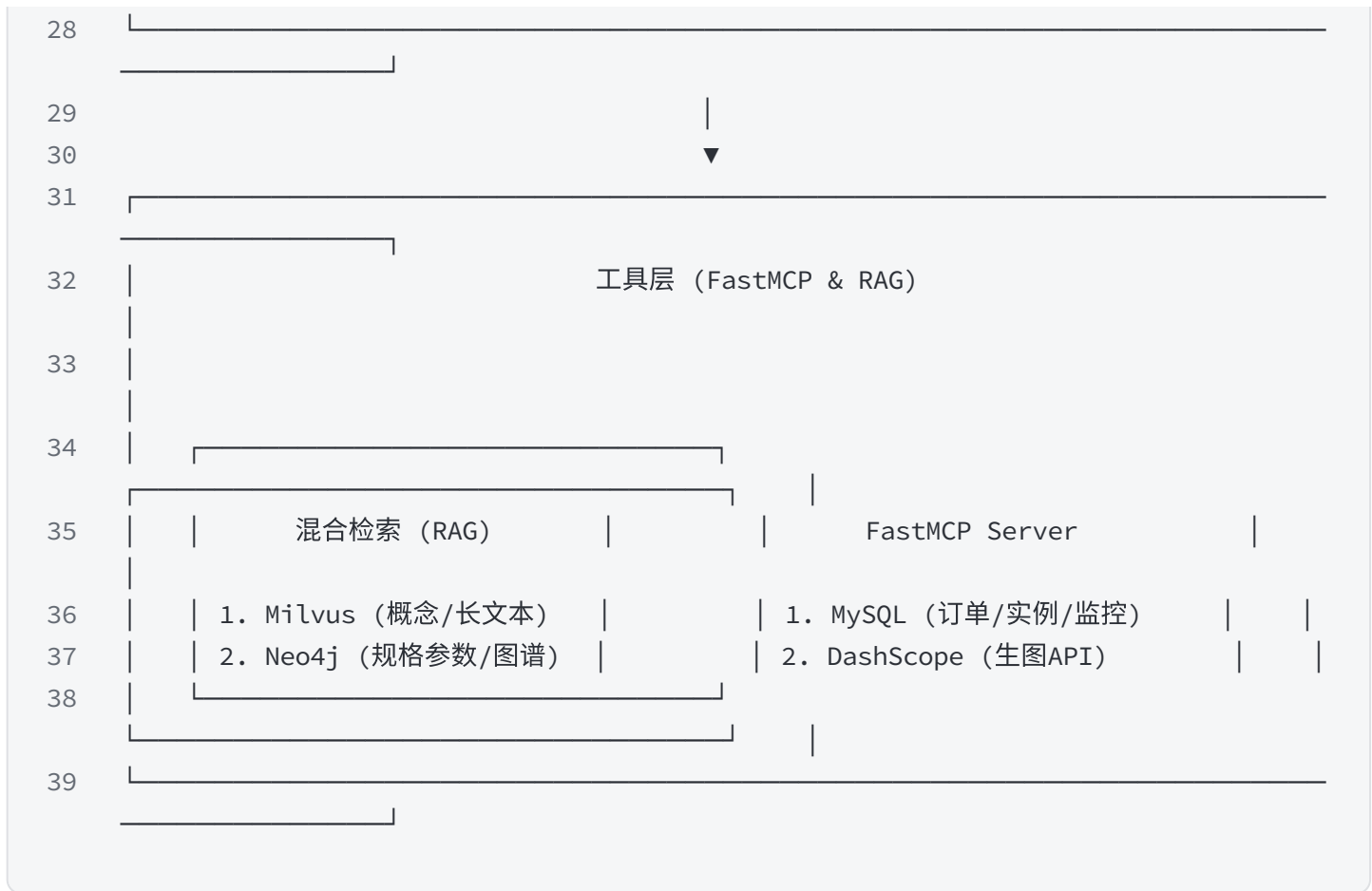
2.1 整体架构图

CloudAgent 的架构从上到下分为四层：接入层、编排层、记忆层与工具层。

代码块

1





2.2 核心模块与代码深度拆解

1. 语义缓存层 (Semantic Cache)

- **位置**: 位于 FastAPI 路由网关，是抵挡大流量的第一道防线。
- **机制**: 基于 DashScope 将用户 Query 向量化，去 Milvus (`qa\_semantic\_cache` 集合) 做检索。
- **核心代码片段** (`app/infra/cache.py`):

```
代码块

1  # 根据向量距离实现 L1 精确命中与高置信度命中
2  async def check_cache(self, query: str) -> str | None:
3      query_vector = await self.get_embedding(query)
4      results = await self.milvus.search(
5          collection_name=self.collection_name,
6          data=[query_vector],
7          anns_field="embedding",
8          param={"metric_type": "COSINE", "params": {"nprobe": 10}},
9          limit=3,
10         output_fields=["question", "answer", "question_norm"]
11     )
12     if not results or not results[0]: return None
13
```

```

14     top_hit = results[0][0]
15     distance = top_hit.distance
16
17     # L1_EXACT: 字符串规范化后完全相等
18     if top_hit.entity.get("question_norm") == _normalize(query):
19         return top_hit.entity.get("answer")
20
21     # L1_SEMANTIC: 语义极度相似 (阈值 0.08)
22     if distance <= 0.08:
23         return top_hit.entity.get("answer")
24
25     return None # 降级, 进入大模型 Agent 推理

```

2. 多智能体编排 (LangGraph)

系统的“大脑”是基于 LangGraph 构建的状态机 (StateGraph)，它通过 `AgentState` 共享上下文，并利用条件边 (Conditional Edges) 实现灵活路由。

- **Orchestrator (路由 Agent)**：不负责回答，只负责根据 `AgentState` 中的 `memory\_context` 和当前 `query`，动态决定下一个执行节点。

- **子 Agent 矩阵**：

- **ProductAgent**：处理政策、参数咨询。挂载 Milvus + Neo4j 双路 RAG。
- **BillingAgent & FinOpsAgent**：执行具体的账单查询与降本分析。
- **RecommendationAgent**：打通咨询到下单闭环。

核心代码片段 (`agent/core/workflow/graph\_manager.py`):

代码块

```

1  # 1. 定义全局状态总线
2  class AgentState(TypedDict):
3      messages: Annotated[list[AnyMessage], operator.add]
4      user_id: str
5      session_id: str
6      memory_context: str # 注入的长短期记忆
7      next_agent: str     # Orchestrator 决定的下一步
8
9  # 2. 构建图结构
10 workflow = StateGraph(AgentState)
11 workflow.add_node("orchestrator", orchestrator_node)
12 workflow.add_node("product_agent", product_agent_node)
13 workflow.add_node("recommendation_agent", recommendation_agent_node)
14 # ... 其他节点
15
16 # 3. 动态条件路由

```

```

17 def route_to_agent(state: AgentState):
18     return state["next_agent"] # 比如返回 "recommendation_agent"
19
20 workflow.add_conditional_edges(
21     "orchestrator",
22     route_to_agent,
23     {
24         "product_agent": "product_agent",
25         "recommendation_agent": "recommendation_agent",
26         "billing_agent": "billing_agent",
27         # ...
28     }
29 )

```

3. MCP 工具化封装 (Model Context Protocol)

传统的 Function Calling 耦合度极高。本项目采用 FastMCP，将底层能力（MySQL 订单查询、DashScope 海报生成）封装为独立的 Server。大模型（Client）通过标准协议与其通信。

- **优势：**物理与逻辑隔离、极强的安全防越权能力。

- **核心代码片段**（`agent/mcp\_servers/cloud\_platform\_server.py`）：

代码块

```

1  from fastmcp import FastMCP
2
3  mcp = FastMCP("CloudPlatformServer")
4
5  # 工具 1: 获取商品库 (带防越权)
6  @mcp.tool()
7  def get_promotable_products(user_id: str) -> str:
8      """获取当前可售卖的云产品目录"""
9      # 业务逻辑...
10     return json.dumps(products)
11
12 # 工具 2: 生成推广海报 (多模态闭环)
13 @mcp.tool()
14 def get_promotion_materials(user_id: str, product_id: str) -> str:
15     """为指定商品生成推广海报和返佣链接"""
16     # 调用 DashScope qwen-image-2.0 API...
17     return f"购买链接: {link}\n海报URL: {image_url}"

```

- ****注入器（`UserIdInjector`）**：**在 LangGraph 节点中，通过拦截器在 LLM 调用工具前，强制把 `state["user\_id"]` 注入进去，防止 LLM 伪造其他人的 ID 查账单。

4. 长短期多级记忆系统 (Memory System)

解决多轮对话中的“失忆”问题。

- **Short-Term (短期)**: Redis 滑动窗口 (`COMPRESSION\_THRESHOLD = 10`), 超过阈值自动裁剪, 防 Context 溢出。

- **Long-Term (长期)**: Milvus 向量库。每次会话结束, 异步触发 LLM 总结用户偏好 (如“偏好高可用架构”) 并存入向量库。

核心代码片段 (`agent/core/memory/manager.py`):

代码块

```
1  class MemoryManager:
2      async def get_context(self, user_id: str, session_id: str) -> str:
3          """会话开始前, 组装记忆上下文"""
4          # 1. 从 Redis 拉取最近对话
5          short_term = await self.short_term.get_history(user_id, session_id)
6          # 2. 从 Milvus 拉取长期偏好
7          long_term = await self.long_term.search_preferences(user_id, limit=3)
8
9          return f"【长期偏好】 \n{long_term}\n【近期对话】 \n{short_term}"
10
11     async def save_conversation(self, user_id: str, session_id: str, ...):
12         """会话结束后, 更新 Redis, 并异步触发长期偏好抽取"""
13         await self.short_term.add_message(...)
14         # 后台抽取偏好存入 Milvus...
```

5. 图向量混合检索 (Hybrid RAG)

为解决架构选型中的参数幻觉, 在 `ProductAgent` 中结合了 Milvus 与 Neo4j。

- 核心代码片段 (`agent/tools/graph\_tool.py` 兜底机制):

代码块

```
1  # 大模型生成的 Cypher 极易报错, 这是关键的容错 Fallback 机制
2  def _fallback_graph_keyword_search(keyword: str) -> str:
3      """当 Cypher 执行失败时, 降级为基于关键词的全文图谱检索"""
4      fallback_query = """
5          MATCH (n)
6          WHERE toLower(n.name) CONTAINS toLower($kw)
7              OR toLower(n.description) CONTAINS toLower($kw)
8          OPTIONAL MATCH (n)-[r]->(m)
9          RETURN n, r, m LIMIT 15
10         """
11     return graph.query(fallback_query, {"kw": keyword})
```

三、核心 Agent 完全拆解

在 LangGraph 框架下，系统被拆分为多个各司其职的 Agent。每个 Agent 都被赋予了特定的人设（System Prompt）、特定的工具链（Tools）以及严格的防幻觉约束。

3.1 Orchestrator (意图路由总控)

核心职责：系统的“大脑调度员”，它不直接回答用户问题，而是根据用户的原始问题和历史记忆，动态决策将任务派发给哪个专业的子 Agent。

设计难点：在多轮对话中，用户的一句“那我选哪个好？”往往缺乏主语，Orchestrator 必须依赖注入的 `memory\_context` 进行指代消解和意图推断。

Prompt 核心代码 (`agent/agents/orchestrator.py`)：

代码块

```
1  system_prompt = f"""你是一个智能客服系统的总路由（Orchestrator）。
2  你的任务是根据用户的提问，决定将问题分发给哪个专业的 Agent 处理。
3
4  当前可用的子 Agent 有：
5  1. "product_agent"：负责云产品介绍、资源规格说明、概念解释等。
6  2. "billing_agent"：负责查询用户个人的云资源实例状态、账单明细等。
7  3. "recommendation_agent"：负责根据用户的业务需求（如Java+MySQL、高并发）提供产品选
   型与推荐。
8  4. "finops_agent_trigger"：当用户表达“账单太贵”、“需要降本增效”、“服务器闲置”等意图时
   选择此项。
9
10  路由细则（高优先级）：
11  - 用户问“某业务场景该选哪个实例”时，必须路由到 recommendation_agent。
12  - 如果你无法判断，默认输出 product_agent。
13
14  【背景记忆】（包含历史对话与长期偏好）：
15  {memory_context}
16
17  请仅输出你要路由到的名称，不要输出任何其他解释性文字。
18  """
```

3.2 Product Agent (产品咨询专家)

核心职责：解决“产品怎么用、规格是什么、支持哪些地域”等事实性问题。

挂载工具：`query\_vector\_db` (Milvus 向量检索)、`query\_knowledge\_graph` (Neo4j 图谱检索)。

防幻觉设计：强制溯源。系统在 Prompt 中下了“死命令”，大模型回答后必须在结尾附上知识库文档的来源。

向量检索实现细节 (`agent/tools/rag\_tool.py`):

代码块

```
1  # 将用户问题转化为向量后, 在 Milvus 中进行 ANN 检索
2  @tool
3  def query_vector_db(query: str) -> str:
4      """查询产品白皮书与政策文档"""
5      query_vector = get_embedding(query)
6      results = milvus_client.search(
7          collection_name="mult_agent_memory",
8          data=[query_vector],
9          anns_field="embedding",
10         param={"metric_type": "COSINE", "params": {"nprobe": 10}},
11         limit=5,
12         output_fields=["text", "source"]
13     )
14
15     # 组装返回给大模型的上下文
16     context = []
17     for hits in results:
18         for hit in hits:
19             context.append(f"[来源:
20 {hit.entity.get('source')}]\\n{hit.entity.get('text')}")
21     return "\\n\\n".join(context)
```

图谱检索实现细节 (`agent/tools/graph\_tool.py`):

为了解决大模型生成的 Cypher 容易出错的问题，我们设计了“大模型生成 Cypher -> 执行 -> 报错兜底 -> 降级关键词搜索”的完整闭环。

代码块

```
1  # 核心执行链路：带有降级机制的图谱检索
2  @tool
3  def query_knowledge_graph(cypher_query: str, fallback_keyword: str) -> str:
4      """执行知识图谱查询, 若 Cypher 失败则使用关键词降级搜索"""
5      try:
6          # 1. 尝试执行大模型生成的标准 Cypher
7          records = graph.query(cypher_query)
8          if records:
9              return format_records(records)
10
11         # 2. 如果执行成功但查无数据, 触发关键词兜底
```

```

12         return _fallback_graph_keyword_search(fallback_keyword)
13
14     except Exception as e:
15         # 3. 如果 Cypher 语法错误导致抛出异常，捕获异常并强制降级
16         print(f"Cypher 执行失败: {e}, 降级为关键词搜索")
17         return _fallback_graph_keyword_search(fallback_keyword)
18
19 def _fallback_graph_keyword_search(keyword: str) -> str:
20     """降级容错方案：使用 CONTAINS 进行字符串模糊匹配"""
21     fallback_query = """
22     MATCH (n)
23     WHERE toLower(n.name) CONTAINS toLower($kw)
24           OR toLower(n.description) CONTAINS toLower($kw)
25     OPTIONAL MATCH (n)-[r]->(m)
26     RETURN n, r, m LIMIT 15
27     """
28     return graph.query(fallback_query, {"kw": keyword})

```

Prompt 核心代码 (`agent/agents/product\_agent.py`):

代码块

```

1  system_prompt = """你是一个专业的云平台产品咨询助手（ProductAgent）。
2  ...
3  【防幻觉警告】
4  你必须基于工具查询到的真实信息回答。绝对不要编造实例型号、参数或编造不存在的文件名。
5
6  【输出格式要求】
7  在每次回答的结尾，你必须严格按照以下格式列出你实际使用的答案来源（如果没有用到某个工具，则
   不要列出对应行）：
8  答案来源：
9  - 向量检索：[实际的文档名，如 product_faq.md]
10 - 图谱检索：[如果有，写"图谱查询"]
11 """

```

3.3 Billing Agent (账单与资产管家)

核心职责：查询用户个人的隐私数据（如名下有几台服务器，最近的订单记录）。

挂载工具：基于 FastMCP 封装的 `query\_user\_instances`、`query\_user\_orders`。

越权防护设计：使用 `UserIdInjector` 拦截器。大模型在调用工具时，工具底层会强制注入当前会话的 `user\_id`，彻底阻断了 Prompt 注入攻击（例如用户输入“帮我查一下 user_id=999 的账单”会失效）。

越权拦截器核心实现 (`agent/agents/billing\_agent.py`):

代码块 基于 LangChain 的 BaseTool 实现动态参数注入

```
2 class UserIdInjector(BaseTool):
3     # 包装原始的 MCP 工具
4     base_tool: BaseTool
5     user_id: str
6
7     def _run(self, *args, **kwargs):
8         # 强制将大模型传入的参数替换/注入为系统真实的 user_id
9         kwargs["user_id"] = self.user_id
10        return self.base_tool._run(*args, **kwargs)
11
12 # 在 Node 节点中动态绑定当前用户的 ID
13 def billing_agent_node(state: AgentState):
14     injected_tools = [
15         UserIdInjector(base_tool=t, user_id=state["user_id"])
16         for t in mcp_tools
17     ]
18     # ... 使用 injected_tools 初始化大模型
```

Prompt 核心代码：

代码块

```
1 system_prompt = """你是一个专业的云平台账单与资源查询助手（BillingAgent）。
2 ...
3 【严格约束】
4 1. 绝对不要编造 instance_id、订单号或金额。
5 2. 只能根据工具返回的数据回答。如果工具返回空，请直接告诉用户“没有查到相关记录”。
6 3. 你的回答必须与工具返回的 JSON 数据 100% 吻合。
7 """
```

3.4 Recommendation Agent (产品推荐与导购专家)

核心职责：打通“技术评估 -> 查库存 -> 生成购买链接”的导购闭环。

执行链路：

1. 调 MCP 工具 `get_promotable_products` 看仓库里哪些机器“真实在售”。
2. 调 RAG 工具 `query_vector_db` 查这些在售机器的“技术说明”（如是否适合跑 Java）。
3. 调 MCP 工具 `get_promotion_materials` 生成这台机器的专属返佣链接和海报。

核心代码逻辑（`agent/agents/recommendation\_agent.py`）：

代码块

```
1 # 动态加载工具集：将 RAG 检索工具与 MCP 商品库工具混合挂载
```

```

2  target_tools = ["get_promotable_products", "search_product_catalog",
  "get_promotion_materials"]
3  mcp_tools = [t for t in all_tools if t.name in target_tools]
4
5  # 跨域工具融合：兼具知识检索与业务办理能力
6  tools = [query_vector_db] + mcp_tools
7
8  system_prompt = """你是一个资深的云架构师和智能推荐顾问。
9  ...
10  【推荐流程】
11  1. 首先调用 get_promotable_products 或 search_product_catalog 获取系统当前“真实可售
  且支持推广”的产品。
12  2. 如果用户有具体技术场景（如“高并发”），调用 query_vector_db 查阅技术文档进行参数匹
  配。
13  3. 推荐结论中，针对推荐的商品，必须调用 get_promotion_materials 获取购买/活动链接，并
  在回复中附上。
14  """

```

3.5 FinOps Agent (降本增效专家)

核心职责：充当云架构财务优化师。针对利用率极低的机器，给出降配（降本）建议。

执行链路（典型的 ReAct 多步推理）：

1. 查用户有哪些实例。
2. 遍历查近 7 天 CPU/内存监控。
3. 分析找出“闲置实例”。
4. 输出量化优化方案。

监控提取与业务闭环代码（`agent/mcp\_servers/cloud\_platform\_server.py`）：

代码块

```

1  # MCP 底层工具封装，直接打通 MySQL 监控表
2  @mcp.tool()
3  def get_instance_metrics(instance_id: str, days: int = 7) -> str:
4      """获取云服务器近期的监控指标（CPU、内存、网络使用率）"""
5      query = """
6          SELECT metric_date, cpu_usage_pct, memory_usage_pct
7          FROM instance_metrics_daily
8          WHERE instance_id = %s AND metric_date >= DATE_SUB(CURDATE(), INTERVAL
  %s DAY)
9      """
10     cursor.execute(query, (instance_id, days))
11     metrics = cursor.fetchall()
12     return json.dumps(metrics, default=str)

```

Prompt 核心代码 (`agent/agents/finops\_agent.py`):

代码块

```
1  system_prompt = """你是一个云平台的降本增效与资源优化专家（FinOpsAgent）。  
2  ...  
3  【分析步骤】  
4  1. 调用 query_user_instances 查询用户当前的实例。  
5  2. 针对每个实例，调用 get_instance_metrics 获取近 7 天的 CPU、内存、带宽使用率。  
6  3. 如果 CPU 连续多天低于 10%，判定为闲置资源，给出具体的“降配建议”（如从 8核 降到 4核）  
   或“关停建议”。  
7  4. 量化收益：必须告诉用户这样做预计每个月能省多少钱。  
8  """
```

四、核心业务数据流 (Data Flow)

以一个典型的“推荐产品并生成海报”的对话为例：

1. **用户输入：**“我是Java服务+MySQL，预算有限，帮我推荐下配置，顺便生成一张海报去给老板看。”
2. **前置缓存：**FastAPI 网关去 Milvus 查缓存，发现是复杂定制需求，未命中，放行至 LangGraph。
3. **记忆注入：**`MemoryManager` 捞出该用户的 Redis 历史对话和 Milvus 偏好，拼装为 `memory\_context` 注入图状态。
4. **意图路由：**`Orchestrator` 判断为选型推荐与推广需求，将 `next\_agent` 设为 `RecommendationAgent`。
5. **工具调用 (MCP & RAG)：**
 - `RecommendationAgent` 调用 MCP 的 `get_promotable_products` 获取当前可售商品清单。
 - 调用 `query_vector_db` 查阅 Java 适合的实例族说明。
 - 确定推荐 `ecs.g8a.xlarge` 后，调用 MCP `get_promotion_materials` 拿到购买链接。
6. **海报生成：**状态可能流转或通过工具直接调用 DashScope 的图生图/文生图 API，生成专属海报 URL。
7. **结果返回与记忆存储：**流式（SSE）返回结果给前端，同时后台触发 Redis 记忆更新。